# Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Cloud Computing Mini-Project

Indian Institute of Information Technology Manipur

**Name:** Retesh Ravindra Mourya
**Roll Number:** 230101020
**Course:** CS3081 – Cloud Computing
**Repository:** GitHub Main Repository

November 2026

## Abstract

This project develops a lightweight natural-language command parser using sequence-level knowledge distillation and deploys it in a containerized Kubernetes environment. The workflow includes teacher-based data generation, dataset cleaning, student-model training, evaluation, object storage, network configuration, CI validation, and performance testing.

The student model achieved 42% exact-match accuracy and 100% JSON validity on the recorded validation dataset. The Task 7 load test processed 284 requests with zero observed failures, with p50 latency of 6400 ms and p95 latency of 8200 ms. The results demonstrate the implementation while highlighting accuracy and latency limitations.

## Tasks Covered

1. Object Storage and Teacher Labelling
2. Student Model Training and Evaluation
3. Containerized Inference Service
4. Kubernetes Deployment and Reliability
5. Subnetting and Network Access Control
6. Reproducibility and CI/CD
7. Performance, Cost, and SLO

# 1 Task 1: Object Storage and Teacher Labelling

## 1.1 Architecture and Implementation

The data pipeline uses SeaweedFS as S3-compatible object storage and Ollama to serve the teacher model. The original command pool contains 3000 records.

Two workers generate teacher-labelled JSONL shards using modulo-based sharding:

$$Shard(i) = i \bmod 2$$

Each worker processes 1500 assigned records. The cleaning script validates JSON syntax, required keys, allowed intents, slot values, and duplicate commands.

## 1.2 Results

| Dataset | Records |
|---|---|
| Original command pool | 3000 |
| train.jsonl | 2426 |
| train_v2.jsonl | 2787 |
| train_v3.jsonl | 2950 |
| Validation dataset | 100 |

*Table 1: Dataset sizes available in the repository.*

The final dataset contains 2950 records. Dataset versions and checksum files are maintained in the repository.

## 1.3 Limitations and Improvements

Structural validation does not guarantee semantic correctness of teacher-generated labels. A valid JSON object can still contain an incorrect intent or slot.

Future improvements include category-wise rejection reporting, semantic validation, and additional manual review of teacher-generated samples.

## 2 Task 2: Student Model Training and Evaluation

### 2.1 Implementation

The student model is EleutherAI/Pythia-70M. Sequence-level knowledge distillation uses complete teacher-generated JSON responses as training targets. The student learns to map natural-language commands to the four fields: intent, date, time, and place.

| Parameter | Configuration |
|---|---|
| Student model | Pythia-70M |
| Learning rate | 0.00005 |
| Batch size | 8 |
| Maximum sequence length | 128 |
| Random seed | 42 |
| Device | CPU |

*Table 2: Recorded training configuration.*

### 2.2 Results

The recorded training loss decreased from approximately 0.6046 to 0.000688 in Version 1 and from approximately 0.5966 to 0.000828 in Version 2.

| Model | Exact | JSON | Intent | Date | Time | Place |
|---|---|---|---|---|---|---|
| Teacher | 26% | 100% | 77% | 72% | 66% | 63% |
| Student v1 | 42% | 100% | 78% | 79% | 84% | 83% |

*Table 3: Validation results on 100 examples.*

The student improved exact-match accuracy from 26% to 42% and maintained 100% JSON validity.

### 2.3 Limitations and Improvements

Training loss does not guarantee generalization. The validation set is small, and teacher errors may propagate to the student.

Future work includes increasing dataset diversity, improving semantic correctness, and evaluating on a larger held-out dataset.

## 3 Task 3: Containerized Inference Service

### 3.1 Architecture and Implementation

The model is served through a FastAPI application inside a Docker container. Model artifacts are downloaded from SeaweedFS during startup.

The service exposes the following endpoints:

- `POST /parse`: Command parsing.
- `GET /healthz`: Liveness check.
- `GET /readyz`: Model readiness check.

The generated output is validated against the required JSON structure. Configuration and object-storage credentials are provided through environment variables and Kubernetes Secrets.

### 3.2 Measured Results

The container was tested with a CPU limit of one core and a memory limit of 1 GiB.

| **Metric** | **Result** |
|---|---|
| Requests | 50 |
| Successful requests | 50/50 |
| OOM kill | No |
| Observed memory | Approximately 385.6 MiB |
| Memory limit | 1 GiB |

*Table 4: Container resource test.*

### 3.3 Limitations and Improvements

The test used sequential requests and does not establish maximum concurrency. Memory usage was sampled rather than continuously measured.

Future work includes concurrent testing, continuous resource monitoring, and inference optimization.

# 4 Task 4: Kubernetes Deployment and Reliability

## 4.1 Implementation

The service is deployed on a local k3d Kubernetes cluster. A Deployment, Service, and Ingress provide application deployment and access. Readiness and liveness probes use the application health endpoints.

| Configuration | Value |
|---|---|
| Replicas | 2 |
| CPU request / limit | 500m / 2 vCPU |
| Memory request / limit | 512 MiB / 1 GiB |
| HPA minimum replicas | 1 |
| HPA maximum replicas | 2 |
| Target CPU utilization | 70% |

*Table 5: Kubernetes configuration.*

## 4.2 Measured Reliability Results

| Test | Requests | Failures | Median | p95 |
|---|---|---|---|---|
| Load | 235 | 0 | 430 ms | 1100 ms |
| Recovery | 374 | 2 | 4000 ms | 8300 ms |
| Rolling update | 660 | 6 | 3900 ms | 6300 ms |
| Rollback | 415 | 6 | 4100 ms | 6400 ms |

*Table 6: Recorded Locust reliability results.*

## 4.3 Limitations and Improvements

The local cluster does not provide multi-machine fault tolerance. The available HPA load-test CSV contains zero requests, so successful measured autoscaling is not established.

Future work includes multi-node deployment and testing HPA scale-up and scale-down under sustained load.

# 5 Task 5: Subnetting and Network Access Control

## 5.1 Subnet Design

The address block 10.20.0.0/16 is divided into four subnets.

| Subnet | Network | Prefix | Usable Hosts |
|---|---|---|---|
| App | 10.20.0.0 | /22 | 1022 |
| Public | 10.20.4.0 | /24 | 254 |
| Data | 10.20.5.0 | /25 | 126 |
| Management | 10.20.5.128 | /27 | 30 |

*Table 7: Subnet allocation.*

Usable hosts are calculated using:

$$UsableHosts = 2^h - 2$$

where $h$ represents the number of host bits.

## 5.2 NACL Implementation

The public subnet is 10.20.4.0/24. The NACL contains rules to deny traffic from 198.51.100.0/24, allow HTTP and HTTPS, permit SSH from the management subnet, and allow required application and reply traffic.

NACLs are stateless. Inbound and outbound reply traffic must be explicitly allowed. Rules are evaluated in ascending order, and the first matching rule determines the action.

## 5.3 Limitations and Improvements

The configuration is documented using subnet and packet sample files. The available evidence represents the configured rules and sample packet evaluation.

Future improvements include testing actual traffic paths, validating rule behavior with automated tests, and reviewing least-privilege access.

# 6 Task 6: Reproducibility and CI/CD

## 6.1 Implementation

The repository includes a Makefile for build, deployment, and teardown operations. The GitHub Actions workflow `.github/workflows/task6.yml` includes:

- Repository checkout and dependency installation.
- Ruff linting.
- Unit tests.
- Model evaluation.
- JSON-validity quality gate.
- Gitleaks secret scanning.

The quality gate requires at least 95% JSON validity. The repository also includes `AI_USE.md`, which documents AI usage.

## 6.2 Results and Evidence

The workflow provides automated validation of the application and model. The repository contains tests for valid and invalid JSON extraction and schema validation.

The assignment requires a deliberately failed quality-gate run using a bad model. The following branch is provided for the failed workflow evidence:

Failed Quality-Gate Evidence

## 6.3 Limitations and Improvements

The branch link identifies the quality-gate demonstration branch; it is not a direct GitHub Actions run URL. The specific failed-run URL should be included if required by the submission instructions.

Future improvements include automated artifact versioning, stronger integration tests, and reproducible deployment validation.

# 7 Task 7: Performance, Cost, and SLO

## 7.1 Measured Load-Test Results

Locust was configured with 20 users, a spawn rate of 2 users/second, and a duration of 2 minutes.

| Metric | Measured Result |
|---|---|
| Requests | 284 |
| Failures | 0 |
| Error rate | 0% |
| Throughput | 2.395 RPS |
| p50 latency | 6400 ms |
| Average latency | 6479.30 ms |
| p95 latency | 8200 ms |
| Maximum latency | 9011.27 ms |
| Average response size | 59 bytes |

*Table 8: Task 7 load-test measurements.*

## 7.2 Resource Usage and Cold Start

| Pod | CPU | Memory |
|---|---|---|
| Pod 1 | 3m | 368 MiB |
| Pod 2 | 3m | 425 MiB |

*Table 9: Point-in-time resource snapshots.*

Cold-start measurements were 18.096, 12.973, and 12.748 seconds. The median was 12.973 seconds and the mean was 14.606 seconds.

## 7.3 Cost Estimate

Using the hypothetical rate card of 3.50/vCPU-hour and 0.45/GiB-hour, one replica with limits of 2 vCPU and 1 GiB memory costs:

$$C_{hour} = (2 \times 3.50) + (1 \times 0.45) = 7.45$$

At the measured throughput:

$$C_{1000} = \frac{7.45}{2.39502275 \times 3600} \times 1000 \approx 0.864$$

This is an estimate, not an actual cloud bill.

## 7.4 SLO and Error Budget

The SLO requires 99% request success.

$$ErrorBudget = 284 \times (1 - 0.99) = 2.84$$

Zero failures were observed, so the error budget used was zero requests.

## 7.5 Limitations and Improvements

The p95 latency of 8200 ms is high. CPU and memory measurements are point-in-time snapshots, and the cost estimate uses hypothetical rates.

Improvements include optimizing generation, gathering continuous metrics, conducting longer load tests, and validating actual cloud costs.

## References


[1] Y. Kim and A. M. Rush, "Sequence-Level Knowledge Distillation," EMNLP, 2016. https://aclanthology.org/D16-1139/

[2] EleutherAI, "Pythia-70M Model." https://huggingface.co/EleutherAI/pythia-70m

[3] Kubernetes Documentation. https://kubernetes.io/docs/

[4] SeaweedFS Documentation. https://github.com/seaweedfs/seaweedfs

[5] Hugging Face Transformers Documentation. https://huggingface.co/docs/transformers